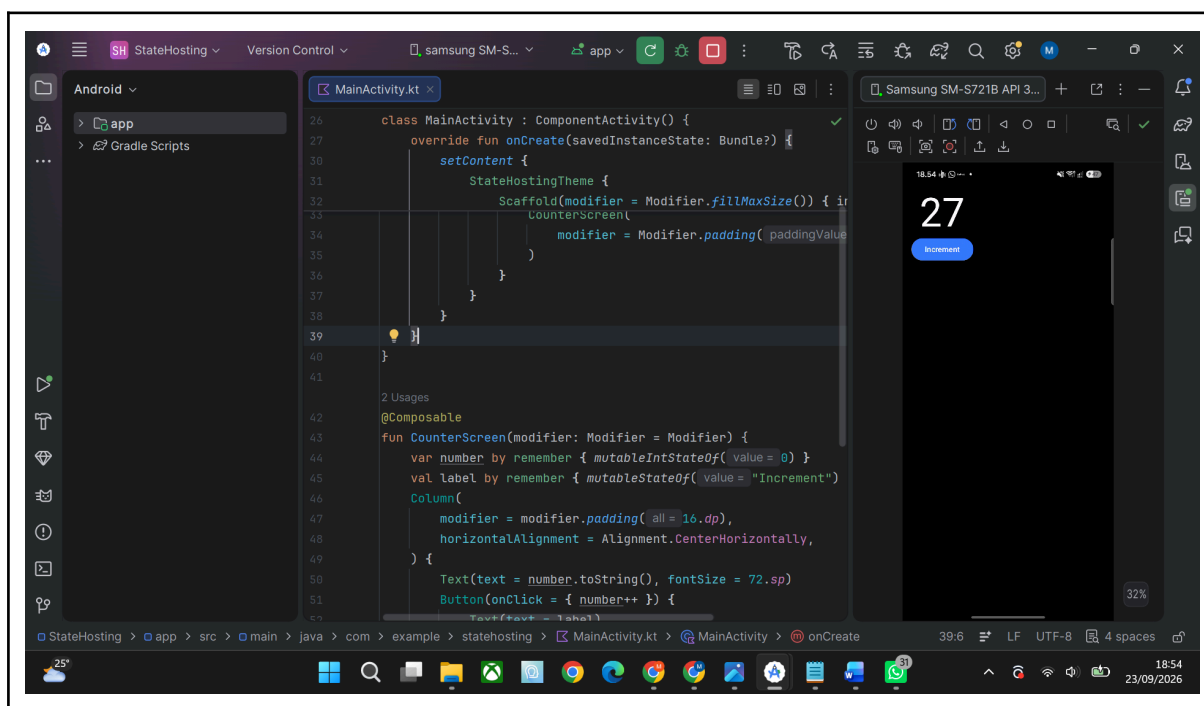


Nama : Muhammad Rifa Aqilla
NIM : 245150407111047
Kelas : Sistem Informasi - C

LAPORAN Modul 4 State Hoisting, Perubahan Konfigurasi, dan Side-effect

4.5.1 State Hoisting

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Nilai angka yang ditampilkan pada Composable Text di atas tombol akan bertambah 1 secara bertahap (misalnya: 0 → 1 → 2 → 3, dst.).

b. Apa yang menyebabkan hal tersebut terjadi?

Saat tombol diklik, lambda onClick pada Button mengeksekusi perintah number++, yang menambahkan nilai variabel number sebesar 1. dan number dideklarasikan menggunakan Compose State (mutableIntStateOf), Jetpack Compose melacak perubahan nilai tersebut dan secara otomatis memicu Recomposition (pembaruan/penggambaran ulang UI) pada bagian Composable yang membaca nilai number.

c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Variabel state number
(Dideklarasikan dengan `var number by remember { mutableIntStateOf(0) }`).

d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

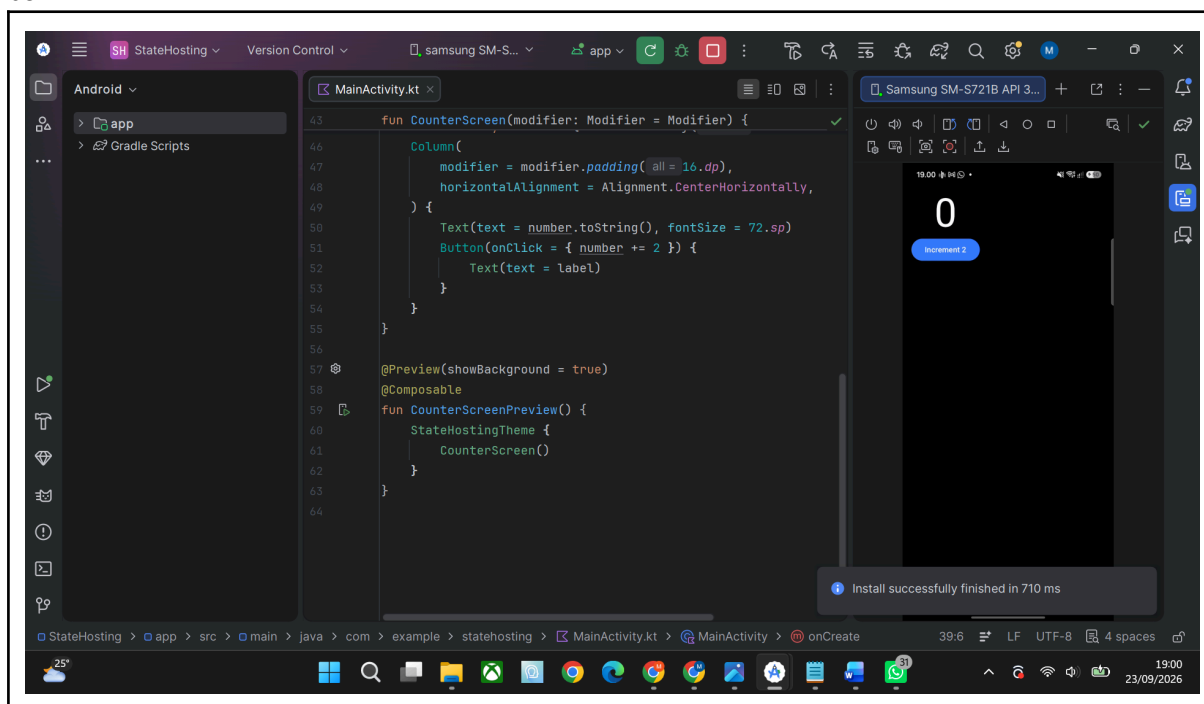
Variabel state label
(Dideklarasikan dengan `val label by remember { mutableStateOf("Increment") }`).

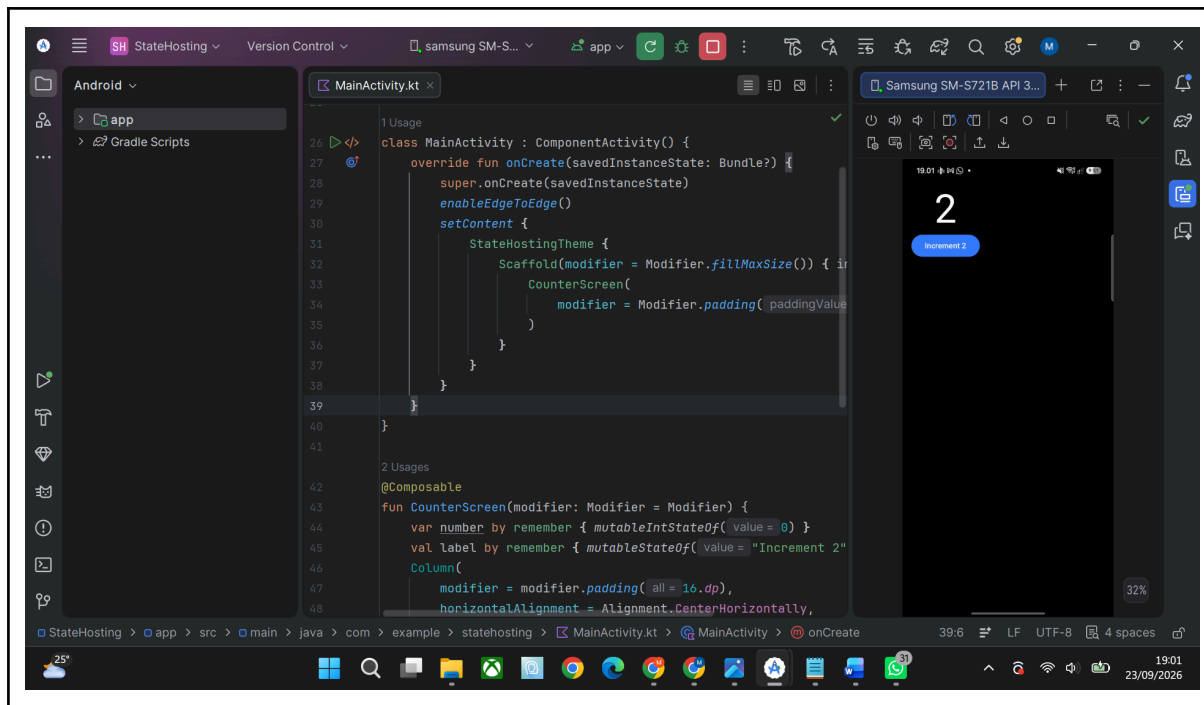
5. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Label Tombol: Mengubah inisialisasi state label dari "Increment" menjadi "Increment 2".
Aksi Tombol: Mengubah logika pada onClick dari `number++` menjadi `number += 2` agar nilai bertambah 2 setiap kali tombol diklik

b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.





7. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, program berjalan dengan baik dan tampilannya persis sama dari sudut pandang pengguna (User Interface). Angka pada layar akan bertambah 1 setiap kali tombol "Increment" diklik. Meskipun struktur kodenya berubah menjadi stateless (state dipindahkan ke pemanggilnya), perilaku aplikasinya tetap identik.

b. Di bagian kode program manakah saat ini proses increment dilakukan?

proses increment saat ini dilakukan di dalam fungsi onCreate() (di blok setContent), tepatnya pada parameter onClick saat memanggil CounterScreen:
 onClick = { number++ }
 Saat tombol diklik, Button di CounterScreen memanggil callback onClick(), yang kemudian mengeksekusi perintah number++ di level MainActivity.

c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State Angka (Composable Text): Dideklarasikan di dalam blok setContent pada fungsi onCreate() menggunakan:
 var number by remember { mutableIntStateOf(0) }
 Label Teks Tombol (Composable Button): Dideklarasikan langsung sebagai argumen literal "Increment" pada parameter label saat memanggil CounterScreen di fungsi onCreate(): label = "Increment"

8. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap tadi dilakukan sedemikian rupa sehingga:

- a. Label tombol "Increment" menjadi "Double"
- b. Nilai awal variabel number dari 0 menjadi 1.
- c. Ubah fungsi/lambda `onButtonClick` pada pemanggilan Composable `CounterScreen` di dalam fungsi `onCreate()` sedemikian rupa sehingga yang dilakukan dari sebelumnya: `number++` menjadi menggandakan nilai variabel `number`.
- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

Bagian kode yang berubah:

```
var number by remember { mutableIntStateOf(1) }
```

```
CounterScreen(  
    modifier = Modifier.padding(innerPadding),  
    number = number,  
    label = "Double",  
    onButtonClick = { number *= 2 },  
)
```

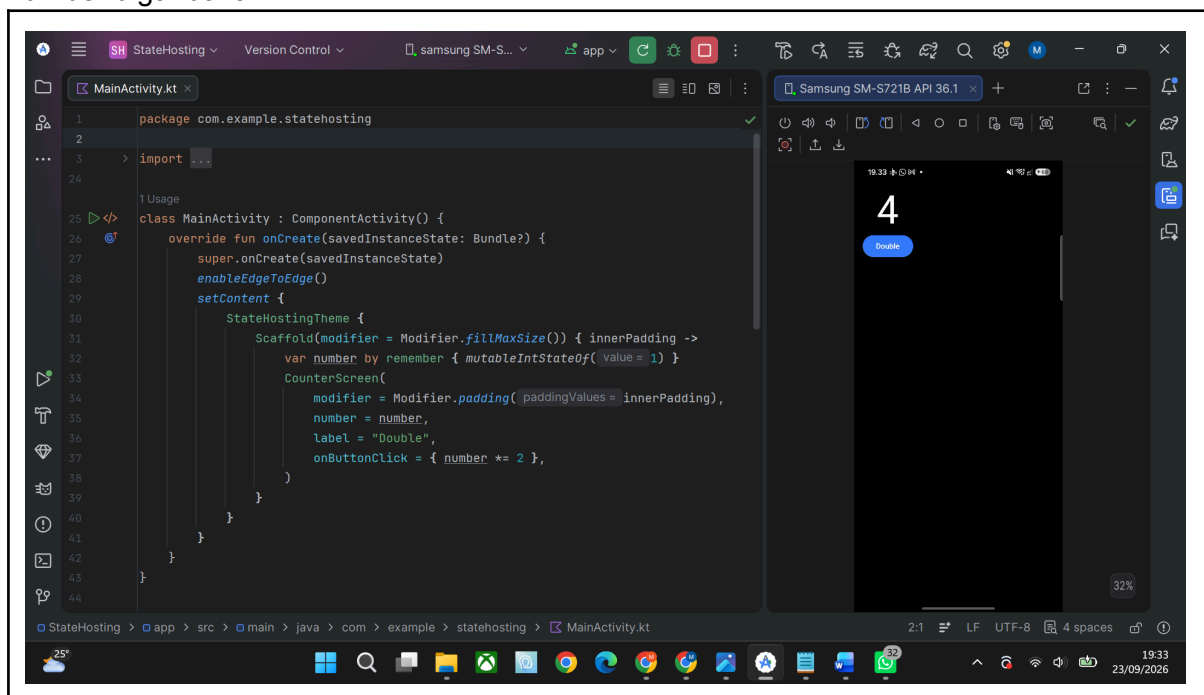
Label Tombol: Diubah dari "Increment" menjadi "Double".

Nilai Awal State: Diubah dari 0 menjadi 1 (`mutableIntStateOf(1)`).

Logika Callback (`onButtonClick`): Diubah dari `number++` menjadi `number *= 2` untuk menggandakan nilai `number` setiap kali tombol diklik (misal: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$, dst.).

9. Jalankan program,

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel `number` digandakan.



b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Pada tahap nomor 7, perubahan dilakukan pada dua bagian, yaitu fungsi CounterScreen dan bagian pemanggilan CounterScreen di dalam onCreate().

Pada tahap nomor 8, perubahan kebutuhan fungsional dilakukan pada bagian onCreate()

Perubahannya meliputi nilai awal menjadi 1, label menjadi "Double", dan operasi tombol menjadi penggandaan nilai.

c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri?

Jika ada sebutkan perubahan yang dilakukan.

Pada tahap nomor 7 terdapat perubahan di dalam CounterScreen, yaitu state number dan label tidak lagi dideklarasikan di dalam fungsi. Selain itu, event tombol juga tidak lagi mengubah number secara langsung.

Namun, pada tahap nomor 8 tidak diperlukan perubahan pada fungsi CounterScreen. Perubahan nilai awal, label tombol, dan logika penggandaan cukup dilakukan melalui parameter saat CounterScreen dipanggil.

10. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

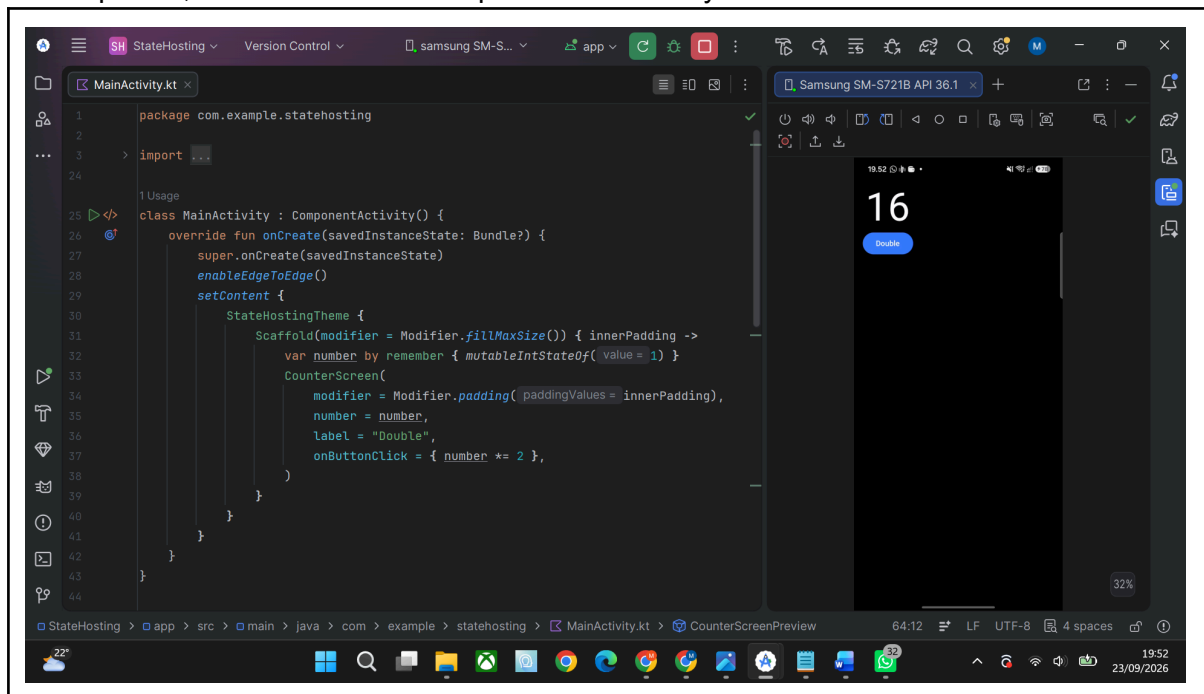
Artinya, state hoisting membuat perubahan kebutuhan fungsional pada UI menjadi lebih mudah dilakukan tanpa harus banyak mengubah Composable CounterScreen.

Pada tahap nomor 9, perubahan dari tombol Increment menjadi Double, nilai awal dari 0 menjadi 1, serta logika dari `number++` menjadi `number *= 2` dilakukan di luar CounterScreen. Karena state dan event sudah dipindahkan ke luar, CounterScreen cukup menerima data dan aksi melalui parameter.

Jadi, manfaat utama state hoisting adalah kode menjadi lebih fleksibel, mudah dipelihara, mudah digunakan kembali, dan lebih mudah disesuaikan ketika kebutuhan aplikasi berubah.

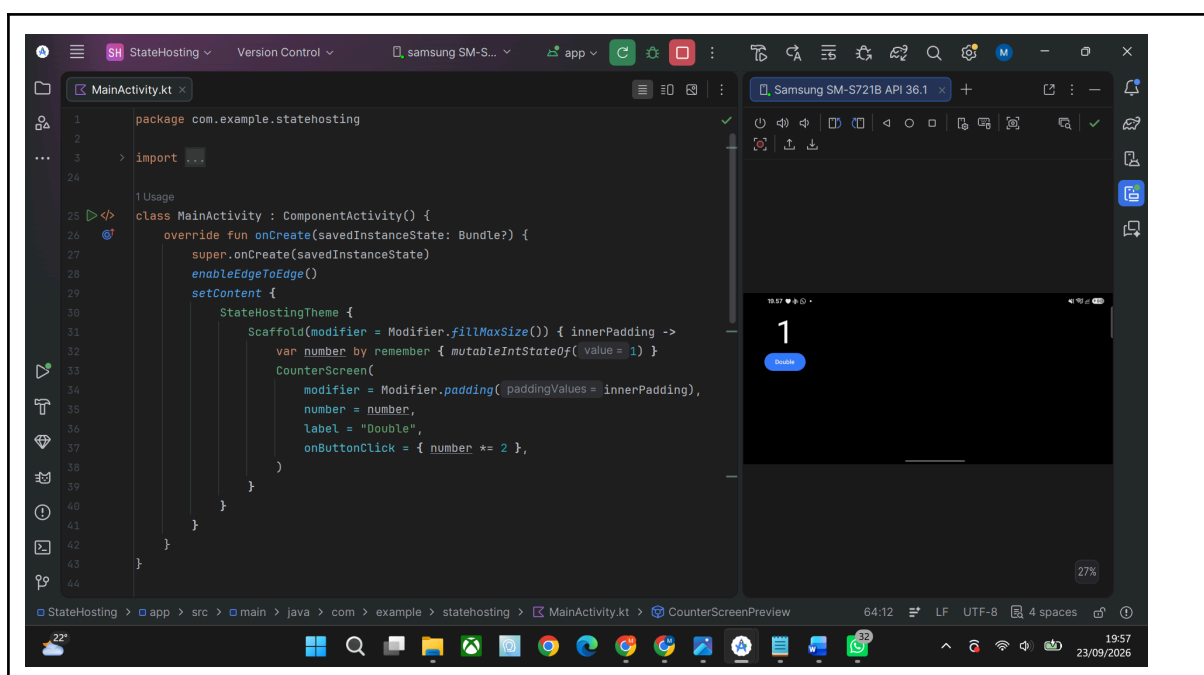
4.5.2 Perubahan Konfigurasi

1. Jalankan program, dengan memastikan posisi perangkat pada posisi *portrait*, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel `number` saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antarmuka aplikasi di posisi *landscape*.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Hal ini terjadi karena sebelumnya state dideklarasikan menggunakan:
`var number by remember { mutableStateOf(1) }`

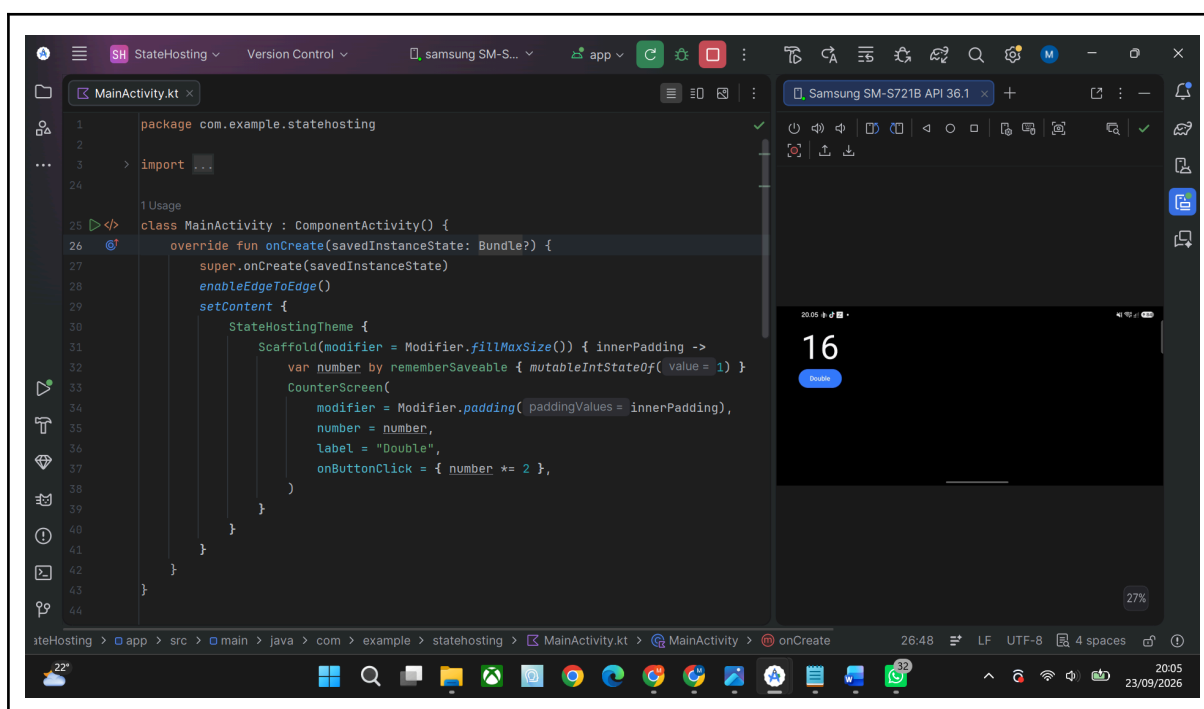
Ketika orientasi perangkat berubah, Android melakukan configuration change. Pada kondisi normal, Activity dibuat ulang sehingga state yang hanya disimpan dengan `remember` dapat hilang.

Akibatnya kode berikut dijalankan kembali:
`mutableStateOf(1)`

sehingga nilai `number` kembali ke nilai awal, yaitu 1.

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antarmuka aplikasi di posisi landscape.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

angka tetap menampilkan angka yang sama ketika pada saat posisi potrait. Karena state sekarang menggunakan:

`var number by rememberSaveable { mutableStateOf(1) }`

`rememberSaveable` menyimpan state sehingga nilainya dapat dipulihkan setelah Activity dibuat ulang akibat perubahan konfigurasi, seperti perubahan orientasi layar.

c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

`remember` digunakan untuk mempertahankan state selama Composable masih berada dalam composition. Namun, state tersebut umumnya tidak bertahan ketika Activity dibuat ulang karena configuration change, misalnya rotasi layar.

Sedangkan **`rememberSaveable`** dapat menyimpan dan memulihkan state setelah perubahan konfigurasi, sehingga nilai tidak hilang ketika perangkat diputar.

d. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Contoh penggunaan `rememberSaveable` dalam aplikasi Android adalah pada form pengisian data, misalnya pengguna sedang mengisi nama atau alamat pada sebuah form. Jika layar berubah dari portrait ke landscape, data yang sudah diketik tetap tersimpan dan tidak kembali kosong.

4.5.3 LaunchedEffect

1. Jalankan Program

a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?

Saat aplikasi pertama kali dijalankan, nilai awal number yaitu 1 akan ditampilkan. Selain itu, akan muncul Snackbar dengan pesan:

Number 1 is shown!

Tombol yang ditampilkan tetap bertuliskan "Double".

b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Pesan tersebut muncul karena `LaunchedEffect` akan langsung dijalankan ketika pertama kali masuk ke dalam proses composition.

c. Apa yang memicu pesan pada `SnackBar` tersebut ditampilkan? Pada bagian kode program yang mana?

Pesan `SnackBar` dipicu oleh `LaunchedEffect(number)`.

Bagian kodenya adalah:
`LaunchedEffect(number) {`

```
    snackbarHostState.showSnackbar("Number $number is shown!")  
  }
```

number berfungsi sebagai key dari LaunchedEffect. LaunchedEffect akan dijalankan ketika pertama kali masuk ke composition dan akan dijalankan kembali setiap kali nilai number berubah.

2. Klik button “Double” satu kali

a. Apa yang terjadi

ketika klik tombol Double angka berubah dari 1 menjadi 2, kemudian menampilkan pesan : Number 2 is shown!

b. Mengapa hal tersebut terjadi? Jelaskan!

Hal tersebut terjadi karena saat tombol Double ditekan, fungsi onClick menjalankan operasi $number *= 2$. Perubahan nilai number menyebabkan antarmuka melakukan recomposition. Karena number digunakan sebagai key pada LaunchedEffect, perubahan nilai tersebut juga menyebabkan LaunchedEffect dijalankan kembali sehingga Snackbar menampilkan nilai terbaru.

Aktivitas Percobaan

1. Jalankan program,

c. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?

Pada saat aplikasi pertama kali dijalankan tanpa menekan tombol Double, layar akan menampilkan nilai awal variabel number, yaitu 1. Selain itu, Snackbar dari percobaan sebelumnya juga dapat menampilkan pesan “Number 1 is shown!”. Hal ini terjadi karena nilai awal number masih ditetapkan sebesar 1 dan LaunchedEffect(number) langsung dijalankan ketika pertama kali masuk ke dalam composition.

d. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Setelah menunggu sekitar dua detik, angka yang sebelumnya bernilai 1 akan berubah menjadi sebuah angka acak antara 100 sampai 999. Angka tersebut berasal dari fungsi Repo.getData(). Setelah nilai number berubah, tampilan pada CounterScreen juga diperbarui dan Snackbar akan menampilkan pesan sesuai angka baru, misalnya “Number 134 is shown!”. Angka yang muncul dapat berbeda setiap kali aplikasi dijalankan karena dihasilkan secara acak.

e. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Perubahan angka setelah dua detik dipicu oleh kode:

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```

Ketika `LaunchedEffect(Unit)` dijalankan, fungsi `Repo.getData()` dipanggil. Di dalam fungsi tersebut terdapat `delay(2000)` yang memberikan jeda selama dua detik, kemudian `Random.nextInt(100, 1000)` menghasilkan angka acak. Hasil tersebut dimasukkan ke variabel `number`.

3. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

`LaunchedEffect(state)` menggunakan suatu nilai `state` sebagai `key`. Artinya, efek di dalamnya akan dijalankan saat pertama kali `Composable` masuk ke `composition` dan akan dijalankan kembali setiap kali nilai `state` tersebut berubah. Contohnya, `LaunchedEffect(number)` akan aktif kembali ketika nilai `number` berubah.

Sedangkan `LaunchedEffect(Unit)` menggunakan `Unit` sebagai `key` yang nilainya tidak berubah. Karena itu, efek di dalamnya umumnya hanya dijalankan satu kali saat `Composable` pertama kali masuk ke dalam `composition`, dan tidak dijalankan ulang hanya karena `state` lain berubah.

b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

Operasi di dalam `LaunchedEffect(state)` dijalankan saat pertama kali `Composable` ditampilkan dan dijalankan kembali setiap kali nilai `state` sebagai `key` berubah. Pada percobaan sebelumnya, `LaunchedEffect(number)` akan berjalan kembali setiap kali nilai `number` berubah, misalnya setelah tombol `Double` ditekan atau setelah data dari `Repo.getData()` diterima.

Sementara itu, operasi di dalam `LaunchedEffect(Unit)` dijalankan ketika `Composable` pertama kali masuk ke `composition`. Pada percobaan, bagian ini digunakan untuk menjalankan `Repo.getData()` satu kali, menunggu dua detik, kemudian mengubah nilai `number` berdasarkan data yang diperoleh.